Muneeb Lone
23i-2623
DS-B

Date: ____/____/____

ALGO HOMEWORK

Selected Problem 1: Cut Ribbon

Objective: The goal is to maximize the number of pieces a ribbon of length n can be cut into, provided that every piece must have a length belongs to a specific set {a, b, c}.

Algorithmic Strategy: Greedy approach fails here because taking the smallest valid cut may repeatedly leave a remainder that cannot be satisfied by any allowed length. We build a table dp[] where dp[i] stores max number of cuts for a ribbon of length i.

DP Transition: Initialize dp[0] = 0 (0 length needs 0 cuts) and all other indices to $-\infty$. For i = 1 to n:

$$dp[i] = 1 + \max(dp[i-a], dp[i-b], dp[i-c])$$

where i ≥ cut length and prev state $\neq -\infty$

Date: ___/___/___

Example:

- n=5
- Allowed cuts: {2,3}

DP Table Computation:

Length (i)	Calculation	dp[i]
0	Base Case	0
1	Impossible	-∞
2	1 + dp[2-2] = 1+0	1
3	1 + dp[3-3] = 1+0	1
4	1 + max(dp[4-2]) = 1+1	2
5	1 + max(dp[5-2], dp[5-3]) = 1+1	2

Result: Max Pieces = 2 (cuts: 2, 3)

Date: ___/___/___

Selected Problem 2: Edit Distance Problem

Objective: To find the minimum number of operations required to convert string A into string B, using only Insert, Delete, Replace. Optimization because minimization problem.

Algorithmic Strategy: We use a 2D DP table. If characters match, no cost is added. If they differ, we take the minimum of the three adjacent states (representing three operations) plus 1.

Recurrence Relation: Let dp[i][j] be the edit distance b/w A[0... i] and B[0 ...j].

$$dp[i][j] = \begin{cases} dp[i-1][j-1] & \text{if } A[i] == B[j] \\ 1 + \min(dp[i][j-1], dp[i-1][j] \\ dp[i-1][j-1]) & \text{if } A[i] \neq B[j] \end{cases}$$

dp[i][j-1] → insert
dp[i-1][j] → delete
dp[i-1][j-1] → replace

CamScanner

Date: ___/___/___

Example: Convert : CAT to CUT

DP Table:

- Row: Source "CAT"
- Col: Target "CUT"

	-	C	U		T	
-	0	1	2		3	
C	1	0	1		2	
A	2	1	1	(Replace A with U)	2	
T	3	2	2		1	(Match T)

Result : Edit Distance is 1.

Date: ___/___/___

Selected Problem 3: Rod Cutting Problem

Objective: Given a rod of length n and a table of prices for various lengths, determine the maximum revenue obtainable by cutting the rod and selling the pieces. Identical to Unbounded Knapsack Problem.

DP Formulation: Let r_n be the max revenue for length n.

$$r_n = \max_{1 \le i \le n} (price[i] + r_{n-i})$$

Example: Rod Length: 4, Price Table:

Len 1 : 1
Len 2 : 5
Len 3 : 8
Len 4 : 9

Computation:

- $r_0 = 0$
- $r_1 = \max(p_1 + r_0) = 1 + 0$
- $r_2 = \max(p_1 + r_1, p_2 + r_0) = \max(1+1, 5+0) = 5$
- $r_3 = \max(p_1 + r_2, p_2 + r_1, p_3 + r_0) = \max(1+5, 5+1, 8+0) = 8$
- $r_4 = \max(p_1 + r_3, p_2 + r_2, p_3 + r_1, p_4 + r_0)$

  $1 + 8 = 9$

  $5 + 5 = 10$ (Max)

  $8 + 1 = 9$

  $9 + 0 = 9$

Max revenue is 10 (cut into two pieces of length 2).

CamScanner

Date: ___/___/___

Selected Problem 4: Coin Change Problem.

Objective: This problem has two variants:

1) Optimization ✓
2) Combinatorics

Recurrence Relation:

$$dp[i] = \min(dp[i], dp[i - coin] + 1)$$

- $dp[0] = 0$, else ∞

Example: Target: 4, Coins: {1, 2}

Table:

- $dp[0] = 0$
- $dp[1] = \min(\infty, dp[0] + 1) = 1$
- $dp[2] = \min(dp[1] + 1, dp[0] + 1) = 1$
- $dp[3] = \min(dp[2] + 1, dp[1] + 1) = 2$
- $dp[4] = \min(dp[3] + 1, dp[2] + 1) = 2$

Result: 2 coins.

CamScanner

5. Fractional Knapsack Problem

Objective: Maximize the total value in a knapsack of capacity W, where items can be broken into smaller fractions.

Why Greedy Works: Unlike the 0/1 Knapsack problem, the fractional version possesses Greedy Choice Property. We can always take item with highest value per weight to fill remaining space.

Strategy:

1. Calculate Ratio, R: $\frac{\text{Value}}{\text{Weight}}$ for all items.
2. Sort items by R in desc order
3. Add whole items while capacity allows.
4. If the next item doesn't fit, take a fraction

$$\frac{\text{Remaining Capacity}}{\text{Item Weight}} \times \text{Item Value}$$

Example: W=50

Item	Value	Weight	Fraction
A	60	10	6
B	100	20	5
C	120	30	4

1. Take A
2. Take B
3. C doesn't fit, take 2/3 of C

Total Value: 60+100+80 = 240